# Trees V (Boosting)

## Math 243

Michael Pearce

# Outline

Last week, we...

- Introduced *classification* trees
- Discussed *ensemble* models, including *bagging* and *random forests*
- Implemented these tree methods in R

In today's class, we will...

- Investigate boosting as another ensemble method for trees
- Implement boosting in R

Section 1

# Boosting for Regression

# Motivation

We saw that Bagging reduces the variance of a model by:

- Taking bootstrap samples of the training data
- Fitting a model (e.g., a regression/classification tree) to each bootstrap sample
- Averaging the predictions from all of the models

# Motivation

We saw that Bagging reduces the variance of a model by:

- Taking bootstrap samples of the training data
- Fitting a model (e.g., a regression/classification tree) to each bootstrap sample
- Averaging the predictions from all of the models

This is great, but it leaves something on the table...

- Each model is built independently of the other models

# Motivation

We saw that Bagging reduces the variance of a model by:

- Taking bootstrap samples of the training data
- Fitting a model (e.g., a regression/classification tree) to each bootstrap sample
- Averaging the predictions from all of the models

This is great, but it leaves something on the table...

- Each model is built independently of the other models

**Boosting** build models sequentially, using information from previous models

- No bootstrapping!
- Instead, fits models to the residuals of the previous model

# Boosting Framework for Regression

1. Fit a model $\hat{f}^{(1)}$ that predicts $Y$ with $X$.

# Boosting Framework for Regression

1. Fit a model $\hat{f}^{(1)}$ that predicts $Y$ with $X$.
2. "Shrink" the model using a small shrinkage parameter $\lambda$ (e.g., $\lambda = 0.01$):

$$\hat{f}^{(1)}(X) \rightarrow \lambda \hat{f}^{(1)}(X)$$

# Boosting Framework for Regression

1) Fit a model $\hat{f}^{(1)}$ that predicts $Y$ with $X$.

2) "Shrink" the model using a small shrinkage parameter $\lambda$ (e.g., $\lambda = 0.01$):

$$\hat{f}^{(1)}(X) \rightarrow \lambda \hat{f}^{(1)}(X)$$

3) Obtain the residuals $R^{(1)}$ from the *shrunken* model,

$$R^{(1)} = Y - \lambda \hat{f}^{(1)}(X)$$

# Boosting Framework for Regression

1. Fit a model $\hat{f}^{(1)}$ that predicts $Y$ with $X$.
2. "Shrink" the model using a small shrinkage parameter $\lambda$ (e.g., $\lambda = 0.01$):
$$\hat{f}^{(1)}(X) \rightarrow \lambda \hat{f}^{(1)}(X)$$
3. Obtain the residuals $R^{(1)}$ from the *shrunken* model,
$$R^{(1)} = Y - \lambda \hat{f}^{(1)}(X)$$
4. Repeat Steps 1-3, but replace $Y$ with residuals $R^{(1)}$.

$$\hat{f}^{(2)}: \text{ predicts } R^{(1)} \text{ with } X$$

# Boosting Framework for Regression

1. Fit a model $\hat{f}^{(1)}$ that predicts $Y$ with $X$.
2. "Shrink" the model using a small shrinkage parameter $\lambda$ (e.g., $\lambda = 0.01$):
$$\hat{f}^{(1)}(X) \rightarrow \lambda \hat{f}^{(1)}(X)$$
3. Obtain the residuals $R^{(1)}$ from the *shrunken* model,
$$R^{(1)} = Y - \lambda \hat{f}^{(1)}(X)$$
4. Repeat Steps 1-3, but replace $Y$ with residuals $R^{(1)}$.

$$\hat{f}^{(2)}: \text{ predicts } R^{(1)} \text{ with } X$$
$$R^{(2)} = R^{(1)} - \lambda \hat{f}^{(2)}(X)$$

# Boosting Framework for Regression

1. Fit a model $\hat{f}^{(1)}$ that predicts $Y$ with $X$.
2. "Shrink" the model using a small shrinkage parameter $\lambda$ (e.g., $\lambda = 0.01$):

$$\hat{f}^{(1)}(X) \rightarrow \lambda \hat{f}^{(1)}(X)$$

3. Obtain the residuals $R^{(1)}$ from the *shrunken* model,

$$R^{(1)} = Y - \lambda \hat{f}^{(1)}(X)$$

4. Repeat Steps 1-3, but replace $Y$ with residuals $R^{(1)}$.

$$\begin{aligned} \hat{f}^{(2)} &: \text{ predicts } R^{(1)} \text{ with } X \\ R^{(2)} &= R^{(1)} - \lambda \hat{f}^{(2)}(X) \\ &= Y - \lambda \left[ \hat{f}^{(1)}(X) + \hat{f}^{(2)}(X) \right] \end{aligned}$$

# Boosting Framework for Regression

1) Fit a model $\hat{f}^{(1)}$ that predicts $Y$ with $X$.

2) "Shrink" the model using a small shrinkage parameter $\lambda$ (e.g., $\lambda = 0.01$):

$$\hat{f}^{(1)}(X) \rightarrow \lambda \hat{f}^{(1)}(X)$$

3) Obtain the residuals $R^{(1)}$ from the *shrunken* model,

$$R^{(1)} = Y - \lambda \hat{f}^{(1)}(X)$$

4) Repeat Steps 1-3, but replace $Y$ with residuals $R^{(1)}$.

$$\hat{f}^{(2)} : \text{ predicts } R^{(1)} \text{ with } X$$

$$\begin{aligned} R^{(2)} &= R^{(1)} - \lambda \hat{f}^{(2)}(X) \\ &= Y - \lambda \left[ \hat{f}^{(1)}(X) + \hat{f}^{(2)}(X) \right] \end{aligned}$$

5) Repeat Steps 1-3 $B$ times (e.g., $B = 1000$). The final model is:

$$\hat{f}(X) = \sum_{b=1}^{B} \lambda \hat{f}^{(b)}(X)$$

# Boosting Framework for Regression: Three Tuning Parameters

1. **Shrinkage Parameter ($\lambda$):** Controls how much we "shrink" the models when updating residuals

$$R^{(b+1)} = R^{(b)} - \lambda \hat{f}^{(b+1)}(X)$$

   - Usually $\lambda \in (0.001,\ 0.1)$.
   - Some values are "better", but not too influential.
   - $\lambda \approx 0.01$ usually works.

# Boosting Framework for Regression: Three Tuning Parameters

1) Shrinkage Parameter ($\lambda$): Controls how much we "shrink" the models when updating residuals

$$R^{(b+1)} = R^{(b)} - \lambda \hat{f}^{(b+1)}(X)$$

- Usually $\lambda \in (0.001,\ 0.1)$.
- Some values are "better", but not too influential.
- $\lambda \approx 0.01$ usually works.

2) Number of Models ($B$): Total number of models in our ensemble.

$$\hat{f}(X) = \sum_{b=1}^{B} \lambda \hat{f}^{(b)}(X)$$

- Chosen via cross-validation

# Boosting Framework for Regression: Three Tuning Parameters

1) Shrinkage Parameter ($\lambda$): Controls how much we "shrink" the models when updating residuals

$$R^{(b+1)} = R^{(b)} - \lambda \hat{f}^{(b+1)}(X)$$

- Usually $\lambda \in (0.001,\ 0.1)$.
- Some values are "better", but not too influential.
- $\lambda \approx 0.01$ usually works.

2) Number of Models ($B$): Total number of models in our ensemble.

$$\hat{f}(X) = \sum_{b=1}^{B} \lambda \hat{f}^{(b)}(X)$$

- Chosen via cross-validation

3) Depth of each tree ($d$): Number of splits in each fitted tree, $\hat{f}^{(b)}$.
- $d$ is often called the interaction depth
- Often $d = 1$ performs well. Each model only uses one predictor.

# Example by-hand

We return to the pdxTrees data set:

```
names(my_pdxTrees)
```

```
## [1] "DBH"                    "Condition"
## [3] "Tree_Height"            "Crown_Width_NS"
## [5] "Crown_Width_EW"         "Crown_Base_Height"
## [7] "Functional_Type"        "Mature_Size"
## [9] "Carbon_Sequestration_lb"
```

```
dim(my_pdxTrees)
```

```
## [1] 3015    9
```

Goal: Predict `Carbon_Sequestration_lb` using all predictors

# Example by-hand (code for demonstration only!!)

Fit the **first** tree to the response variable (with depth $d = 1$):

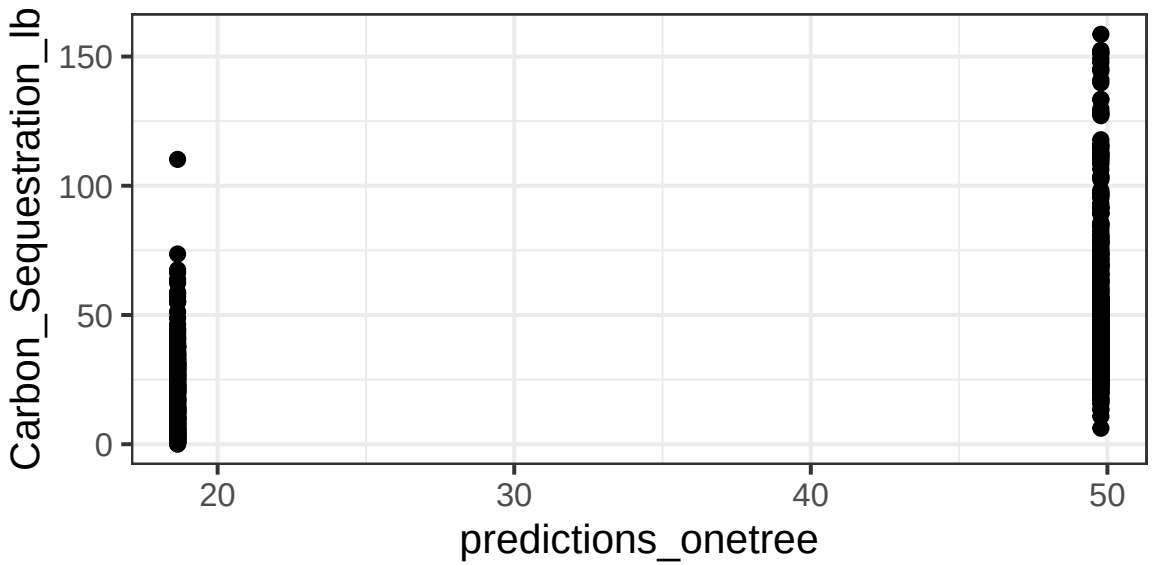
```
boost_tree_model1<- rpart(Carbon_Sequestration_lb ~ .,
                          data = my_pdxTrees_train,
                          control = rpart.control(maxdepth = 1))
```

# Example by-hand (code for demonstration only!!)

Fit the **first** tree to the response variable (with depth $d = 1$):

```
boost_tree_model1<- rpart(Carbon_Sequestration_lb ~ .,
                          data = my_pdxTrees_train,
                          control = rpart.control(maxdepth = 1))
```

Predicted vs. true values in test data:




- Only two possible values because our tree depth is 1 (i.e., one split)

# Example by-hand (code for demonstration only!!)

Get residuals from **first** shrunken model with $\lambda = 0.5$ (too big in practice):

```
shrunken_fitted1 <- 0.5*predict(boost_tree_model1, my_pdxTrees_train)
boost_pdxTrees_train <- my_pdxTrees_train %>%
  mutate(residuals1 = Carbon_Sequestration_lb - shrunken_fitted1) %>%
  select(-Carbon_Sequestration_lb)
```

# Example by-hand (code for demonstration only!!)

Get residuals from **first** shrunken model with $\lambda = 0.5$ (too big in practice):

```
shrunken_fitted1 <- 0.5*predict(boost_tree_model1, my_pdxTrees_train)
boost_pdxTrees_train <- my_pdxTrees_train %>%
  mutate(residuals1 = Carbon_Sequestration_lb - shrunken_fitted1) %>%
  select(-Carbon_Sequestration_lb)
```

Fit the **second** tree to the residuals:

```
boost_tree_model2<- rpart(residuals1 ~ . , data = boost_pdxTrees_train,
                          control = rpart.control(maxdepth = 1))
```

# Example by-hand (code for demonstration only!!)

Get residuals from **first** shrunken model with $\lambda = 0.5$ (too big in practice):

```
shrunken_fitted1 <- 0.5*predict(boost_tree_model1, my_pdxTrees_train)
boost_pdxTrees_train <- my_pdxTrees_train %>%
  mutate(residuals1 = Carbon_Sequestration_lb - shrunken_fitted1) %>%
  select(-Carbon_Sequestration_lb)
```

Fit the **second** tree to the residuals:

```
boost_tree_model2<- rpart(residuals1 ~ . , data = boost_pdxTrees_train,
                  control = rpart.control(maxdepth = 1))
```

Get residuals from **second** shrunken model:

```
shrunken_fitted2 <- 0.5*predict(boost_tree_model2, my_pdxTrees_train)
boost_pdxTrees_train <- boost_pdxTrees_train %>%
  mutate(residuals2 = residuals1 - shrunken_fitted2) %>%
  select(-residuals1)
```

# Example by-hand (code for demonstration only!!)

Get residuals from **first** shrunken model with $\lambda = 0.5$ (too big in practice):

```
shrunken_fitted1 <- 0.5*predict(boost_tree_model1, my_pdxTrees_train)
boost_pdxTrees_train <- my_pdxTrees_train %>%
  mutate(residuals1 = Carbon_Sequestration_lb - shrunken_fitted1) %>%
  select(-Carbon_Sequestration_lb)
```

Fit the **second** tree to the residuals:

```
boost_tree_model2<- rpart(residuals1 ~ . , data = boost_pdxTrees_train,
                  control = rpart.control(maxdepth = 1))
```

Get residuals from **second** shrunken model:

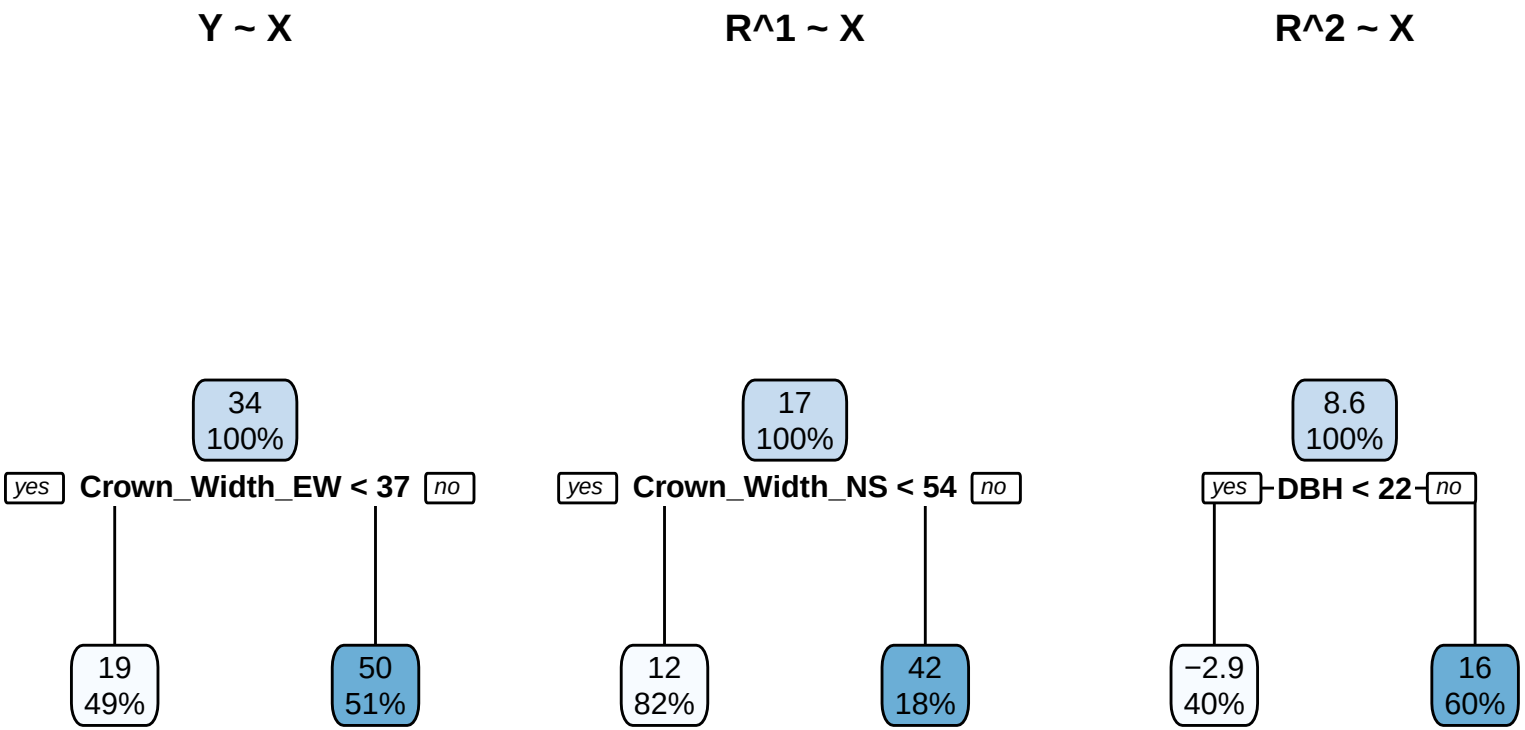
```
shrunken_fitted2 <- 0.5*predict(boost_tree_model2, my_pdxTrees_train)
boost_pdxTrees_train <- boost_pdxTrees_train %>%
  mutate(residuals2 = residuals1 - shrunken_fitted2) %>%
  select(-residuals1)
```

Fit the **third** tree to the *new* residuals:

```
boost_tree_model3<- rpart(residuals2 ~ . , data = boost_pdxTrees_train,
                  control = rpart.control(maxdepth = 1))
```

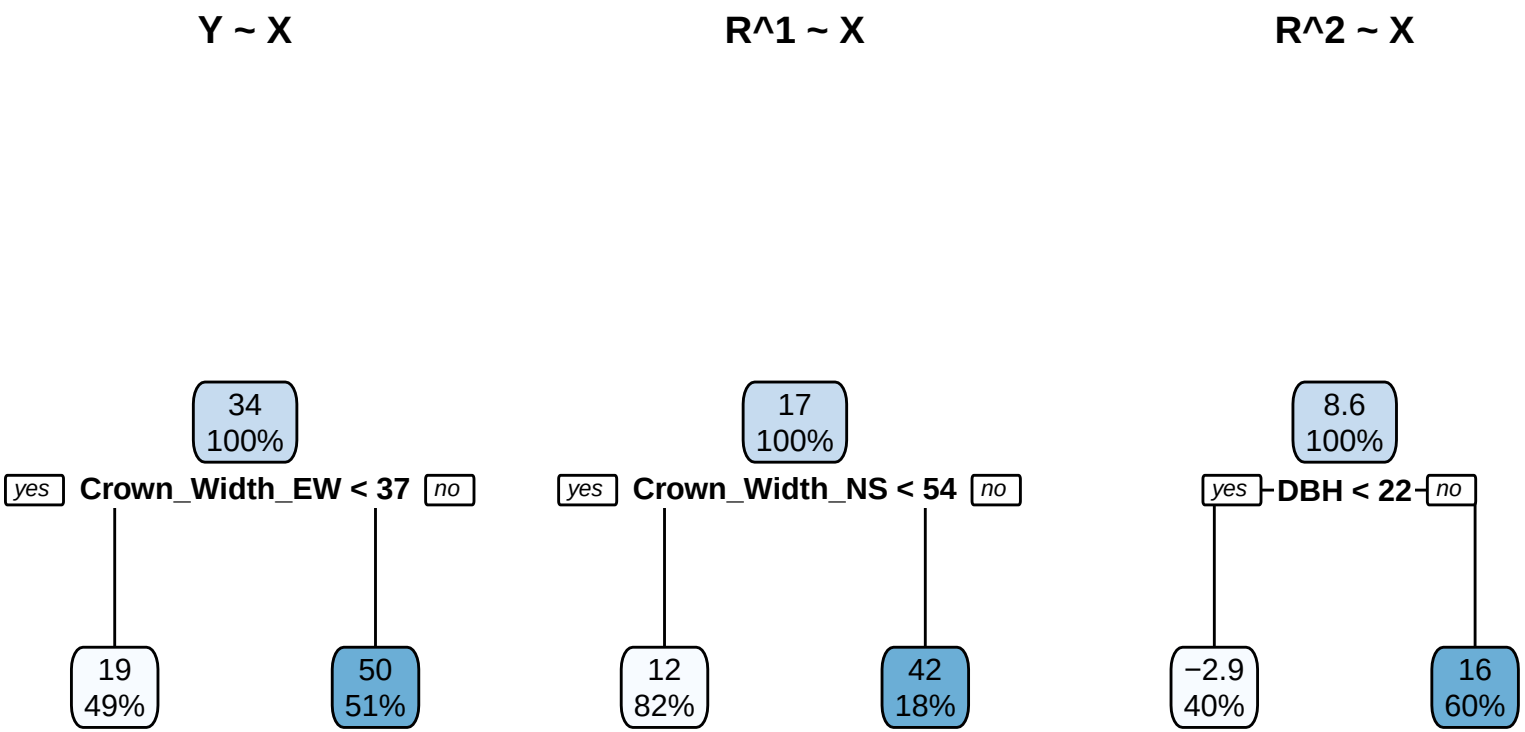
# Example by-hand (code for demonstration only!!)

Let's check out the trees:



# Example by-hand (code for demonstration only!!)

Let's check out the trees:




The average response variable in each tree is decreasing

Q: Why is this happening?

# Example by-hand (code for demonstration only!!)

Combine trees to get predicted values on test data:

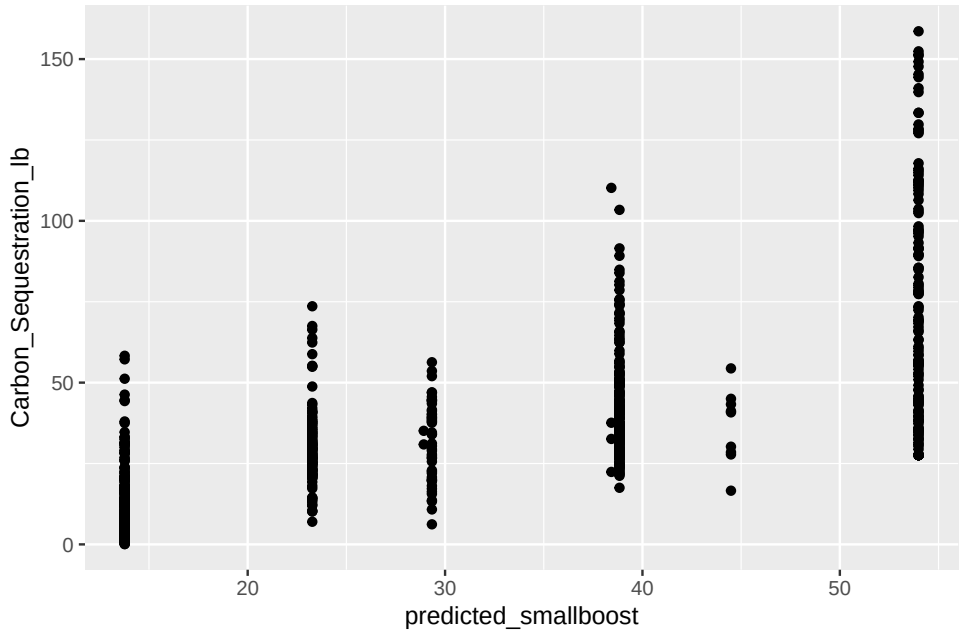
```
shrunken_predicted1 <- 0.5*predict(boost_tree_model1, my_pdxTrees_test)
shrunken_predicted2 <- 0.5*predict(boost_tree_model2, my_pdxTrees_test)
shrunken_predicted3 <- 0.5*predict(boost_tree_model3, my_pdxTrees_test)
predicted_smallboost <- shrunken_predicted1 + shrunken_predicted2 + shrunken_predicted3
```

# Example by-hand (code for demonstration only!!)

Combine trees to get predicted values on test data:

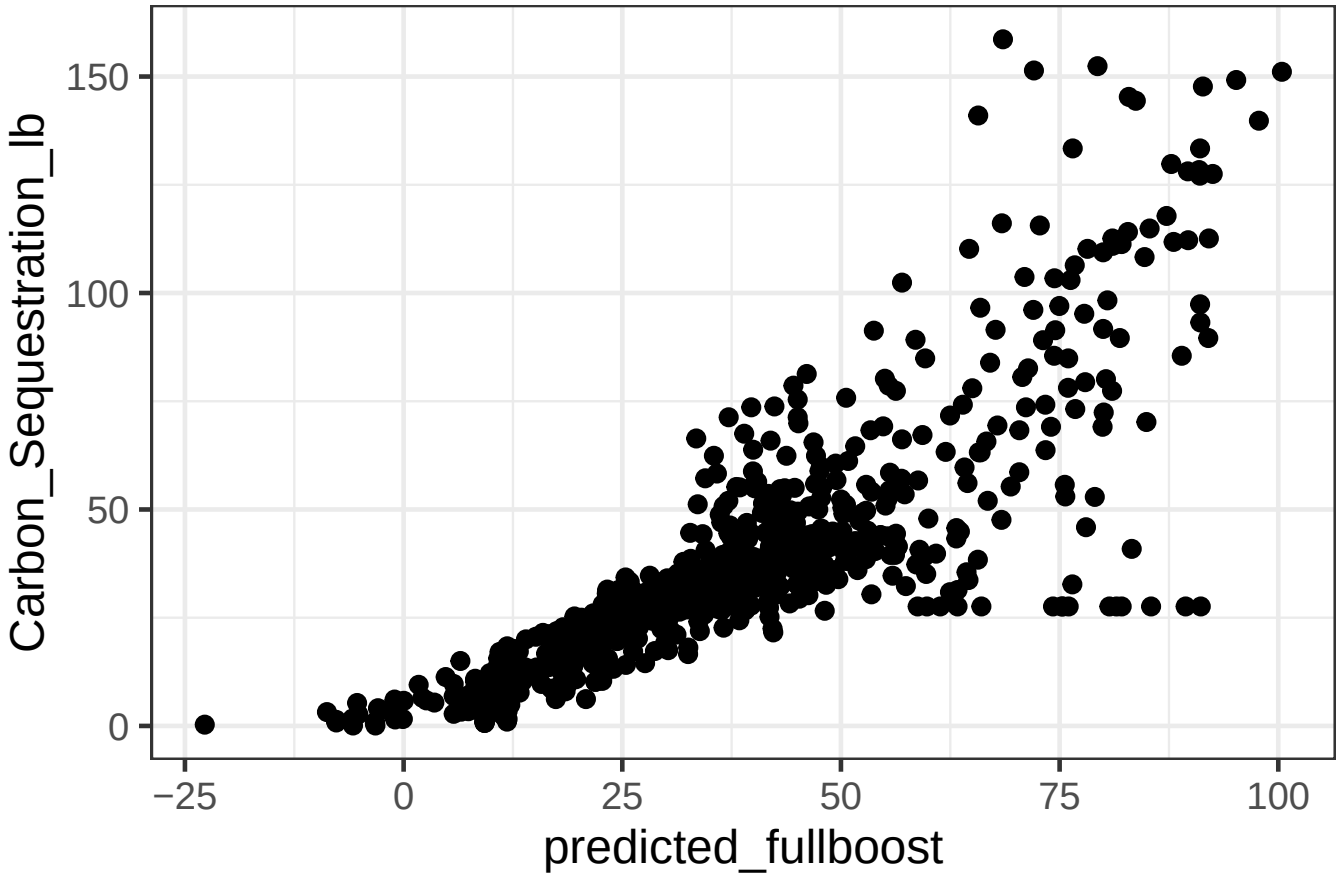
```
shrunken_predicted1 <- 0.5*predict(boost_tree_model1, my_pdxTrees_test)
shrunken_predicted2 <- 0.5*predict(boost_tree_model2, my_pdxTrees_test)
shrunken_predicted3 <- 0.5*predict(boost_tree_model3, my_pdxTrees_test)
predicted_smallboost <- shrunken_predicted1 + shrunken_predicted2 + shrunken_predicted3
```

Predicted vs. true values in test data:




- More possible predicted values because of more trees

# If we did a full boosting algorithm (with a smaller $\lambda$)...



# TPS: Boosting vs. Random Forests

Let's compare Boosting (with trees) and Random Forests:

- Q: What are the similarities?
- Q: What are the differences?

# TPS: Boosting vs. Random Forests

Let's compare Boosting (with trees) and Random Forests:

- Q: What are the similarities?
- Q: What are the differences?

**Similarities:**

- Final prediction is the sum/average of predictions from an ensemble of trees

**Differences:**

- In Random Forests, all trees are created independently, while Boosting trees are dependent on previous trees (through residuals)
- In Random Forests, all trees are of maximum depth, while Boosting trees can be short

# Boosting for Classification

Boosting can't be straightforwardly applied to classification problems:

- Boosting requires "residuals"
- No residuals in classification (Green - Blue = ?)

# Boosting for Classification

Boosting can't be straightforwardly applied to classification problems:

- Boosting requires "residuals"
- No residuals in classification (Green - Blue = ?)

Still, there are boosting methods for classification

- E.g., "AdaBoost" and "Stochastic Gradient Boosting"

# Boosting for Classification

Boosting can't be straightforwardly applied to classification problems:

- Boosting requires "residuals"
- No residuals in classification (Green - Blue = ?)

Still, there are boosting methods for classification

- E.g., "AdaBoost" and "Stochastic Gradient Boosting"

The boosting package we cover uses the latter.

- Idea: Create residuals using the probability of predicting each class
- Probabilities are numbers, and thus can be used to produce residuals

Section 2

# Boosting in R

# Boosting in R with gbm

Can use the gbm function in the gbm package to create Boosted Trees

```
library(gbm)
set.seed(10101)
boosted_tree<-gbm(Carbon_Sequestration_lb ~., my_pdxTrees_train,
                  distribution = "gaussian",
                  n.trees=4000,
                  interaction.depth = 1,
                  shrinkage = .03)
```

# Boosting in R with gbm

Can use the gbm function in the gbm package to create Boosted Trees

```
library(gbm)
set.seed(10101)
boosted_tree<-gbm(Carbon_Sequestration_lb ~., my_pdxTrees_train,
                  distribution = "gaussian",
                  n.trees=4000,
                  interaction.depth = 1,
                  shrinkage = .03)
```

- For regression problems, use the argument `distribution = "gaussian"`
- For classification problems, we use `distribution = "bernoulli"`

# Boosting in R with gbm

Can use the gbm function in the gbm package to create Boosted Trees

```
library(gbm)
set.seed(10101)
boosted_tree<-gbm(Carbon_Sequestration_lb ~., my_pdxTrees_train,
                  distribution = "gaussian",
                  n.trees=4000,
                  interaction.depth = 1,
                  shrinkage = .03)
```

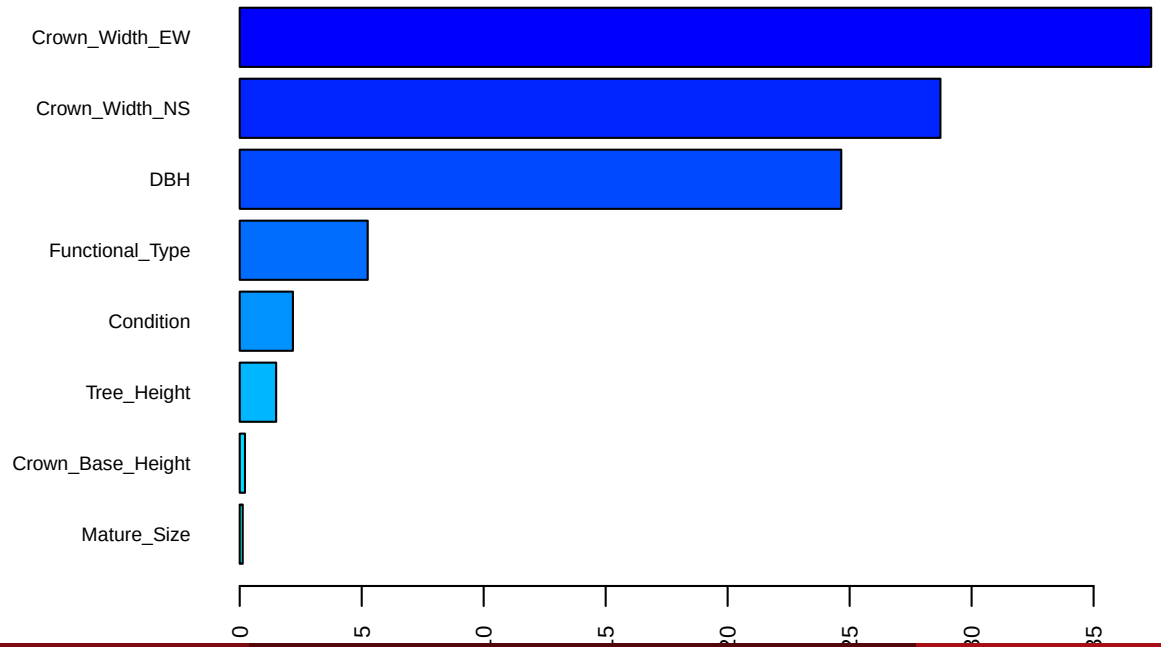
- For regression problems, use the argument `distribution = "gaussian"`
- For classification problems, we use `distribution = "bernoulli"`

- `n.trees` determines the number of trees ($B$)
- `interaction.depth` determines the "depth" of each tree ($d$)
- `shrinkage` determines the shrinkage parameter ($\lambda$)

# Summary Information with summary()

```
summary(boosted_tree, plotit = T)
```

```
##                                 var    rel.inf
## Crown_Width_EW       Crown_Width_EW 37.3606094
## Crown_Width_NS       Crown_Width_NS 28.7225535
## DBH                             DBH 24.6571212
## Functional_Type     Functional_Type  5.2462150
## Condition                 Condition  2.1805164
## Tree_Height             Tree_Height  1.4930932
## Crown_Base_Height Crown_Base_Height  0.2166996
## Mature_Size             Mature_Size  0.1231917
```



# Making Predictions with `predict()`

You can make predictions using the `predict()` function:

```
predict(boosted_tree, my_pdxTrees_test)
```

```
## [1] 45.22739 82.73005 43.88327 31.78899 48.80809
```

# Cross-Validation

The shrinkage parameter ($\lambda$) and tree depth ($d$) are typically chosen by the user

- But the number of trees $B$ can be chosen by cross-validation!

# Cross-Validation

The shrinkage parameter ($\lambda$) and tree depth ($d$) are typically chosen by the user

- But the number of trees $B$ can be chosen by cross-validation!
- Can implement CV and set the number of folds with the `cv.folds` option:

```
library(gbm)
set.seed(10101)
boosted_cv<-gbm(Carbon_Sequestration_lb ~., my_pdxTrees_train,
                distribution = "gaussian",
                n.trees=8000,
                interaction.depth = 1,
                shrinkage = .01,
                cv.folds=10)
```

# Cross-Validation

The shrinkage parameter ($\lambda$) and tree depth ($d$) are typically chosen by the user

- But the number of trees $B$ can be chosen by cross-validation!
- Can implement CV and set the number of folds with the `cv.folds` option:

```
library(gbm)
set.seed(10101)
boosted_cv<-gbm(Carbon_Sequestration_lb ~., my_pdxTrees_train,
                distribution = "gaussian",
                n.trees=8000,
                interaction.depth = 1,
                shrinkage = .01,
                cv.folds=10)
```

CV error can be accessed by adding `$cv.error`:

```
boosted_cv$cv.error
```

```
##  [1] 783.5981 778.9109 774.3375 769.9014 765.4010 761.0370 756.6461 752.3661
##  [9] 747.9905 743.7954
```

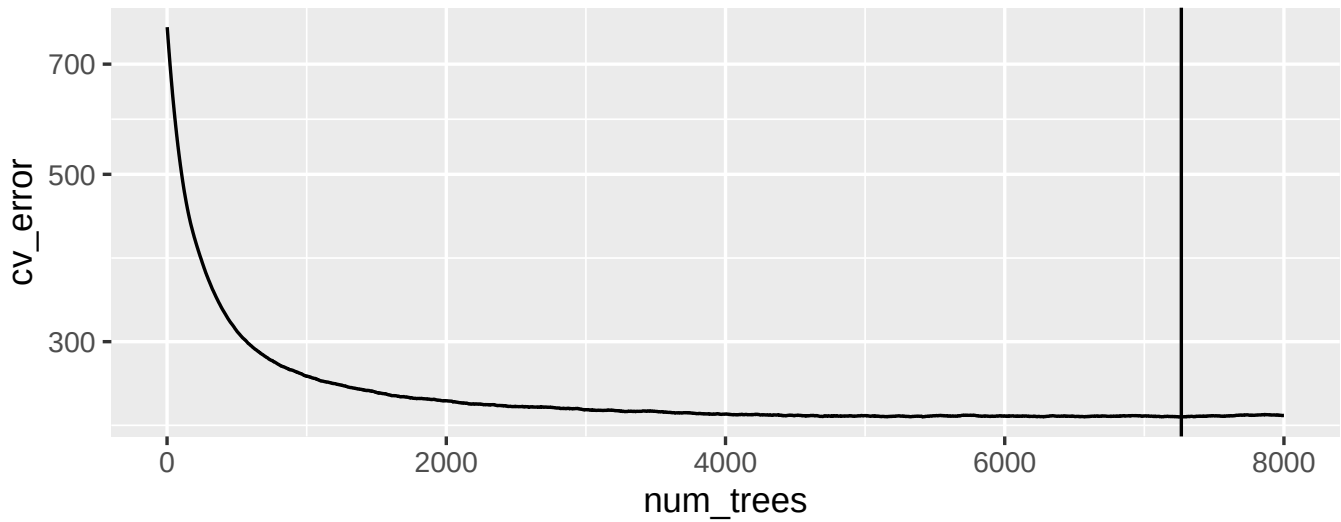
- The length of this vector is the number of trees (here `n.trees`=8000)

# Cross-Validation

```
which.min(boosted_cv$cv.error)
```

```
## [1] 7265
```

```
cv_df <- data.frame(num_trees=1:8000, cv_error = boosted_cv$cv.error)
ggplot(cv_df, aes(x=num_trees, y=cv_error)) + scale_y_log10()+
  geom_line() +  geom_vline(xintercept = 7265)
```




```
boosted_final<-gbm(Carbon_Sequestration_lb ~., my_pdxTrees_train,
                   distribution = "gaussian", n.trees=7265,
                   interaction.depth = 1,shrinkage = .01)
```

# Boosted Tree Performance

How does the boosted tree do vs Random Forest? A bagged tree? A pruned tree? A linear model?

```
## # A tibble: 5 x 4
##   model         .metric .estimator .estimate
##   <chr>         <chr>   <chr>          <dbl>
## 1 bagged_tree   rmse    standard        10.9
## 2 random_forest rmse    standard        11.4
## 3 pruned_tree   rmse    standard        13.7
## 4 boosted_tree  rmse    standard        15.3
## 5 linear_model  rmse    standard        17.7
```

- The boosted tree isn't very good in this context!
- Bagging does best, linear model doesn't worst.

Relative performance of methods depends on the data!

- Try out a few methods – it's quick to fit models in R.

Section 3

# Recap

# Recap

Today, we...

- Learned about Boosting and implemented it in R

Later this week:

- Wednesday: Practice trees in Homework 10
- Thanksgiving Break!